SECURITY FUNDAMENTALS ASSESSMENT REPORT

SWYNEX Technologies – Internship Task 1

Assessment type	Hands-on security practice using authorized PortSwigger Web Security Academy labs
Scope	Three intentionally vulnerable training labs covering authentication, access control and path traversal
Tools	Web browser and Burp Suite Community Edition
Date	3 November 2026

About this report

This report is a write-up of the practical work I did for the first SWYNEX internship task. I worked through three labs on the PortSwigger Web Security Academy, noted what I did at each stage, and then thought about why each weakness exists and how a developer could prevent it. Everything here was done inside PortSwigger's own training environment, which is built to be attacked legally. No real company's website, system or account was tested at any point.

1. Introduction

Web applications fail in surprisingly ordinary ways. A page forgets to check who is asking, a login flow trusts the browser a little too much, or a file name is passed straight to the operating system. None of these needs an exotic exploit, which is exactly why they keep showing up in real breaches.

For this task I wanted to see a few of these problems for myself instead of only reading about them. I created an account on the PortSwigger Web Security Academy, browsed through its learning topics and picked three labs that each show a different kind of weakness. I then solved them one by one, using the browser for most of the work and Burp Suite Community Edition where I needed to look at or change raw HTTP requests.

Alongside the steps I took, this report records what each issue means in practice, how serious it could be, and what a team should do to fix it.

2. Summary of Findings

All three labs were completed and showed the Solved status, as the screenshots in Section 5 confirm. The table below gives a quick overview.

No.	Lab	Category	What went wrong	Suggested severity
1	2FA simple bypass	Authentication	The account page could be opened without ever entering the verification code	High
2	Unprotected admin functionality	Broken access control	An admin panel was reachable by anyone who knew its path, and robots.txt gave the path away	High
3	File path traversal, simple case	Path traversal	The image filename parameter let me read /etc/passwd from the server	High

Severity note: these ratings apply to the lab scenarios as they were set up. In a real application the true severity would depend on how exposed the system is, what data it holds and what other protections exist.

3. Environment, Tools and Approach

Environment. The PortSwigger Web Security Academy is a free platform from the makers of Burp Suite. Its labs are deliberately vulnerable websites that you are allowed to attack, so they are a safe place to learn.

Tools. I used a normal web browser for all three labs. For the path traversal lab I also used Burp Suite Community Edition, mainly its Proxy and HTTP history view, to see the requests my browser was sending and to edit one of them.

Setting up. The preparation was simple:

1. Visited the PortSwigger Web Security Academy website.
2. Created an account and logged in.
3. Went through the learning topics to understand the common web vulnerabilities.
4. Picked three practical labs, one each from authentication, access control and path traversal.

5. Got the browser and Burp Suite Community Edition ready for the exercises.

Limits of this work. This was guided practice, not a full penetration test. I did not review any source code or infrastructure, run automated scanners, or measure real-world impact. The aim was to understand how each weakness works and how it is fixed.

4. Detailed Findings

Finding 1: 2FA Simple Bypass

Category: Authentication vulnerability | **Suggested severity:** High

The goal. The lab asked me to get into Carlos's account without completing the two-factor verification step. The lab gives you two sets of credentials: `wiener:peter` for your own account and `carlos:montoya` for the victim.

What I did.

1. Opened the 2FA simple bypass lab from the Authentication section and read the description.
2. Logged in with my own account (wiener) and opened the lab's email client to read the 2FA code that had been sent.
3. Once logged in, I opened the account page and noted down its URL.
4. Logged out, then logged in with the victim's username and password.
5. When the site asked for the verification code, I did not enter anything. I pasted the account page URL I had saved earlier into the address bar instead.
6. Carlos's account page loaded straight away, with no 2FA code entered, and the lab showed Solved.

Why it works. The application treated a correct password as enough to open the account page. The second step existed on screen, but the server never actually required it before serving the protected page, so skipping the screen skipped the check.

Potential impact. If an attacker already has someone's password, from a leak or a phishing page for example, 2FA is meant to be the thing that stops them. With this flaw it stops nothing, and the account is exposed as if 2FA was never turned on.

Recommended fix. The server should keep track of how far through the login a user is, and should not mark them as fully logged in until the second factor has been verified. Every protected page should check for that fully authenticated state, and half-finished login sessions should expire or be cancelled.

What I took from it: two-factor authentication has to be enforced on the server for every protected page, not just shown as a step in the login flow.

Finding 2: Unprotected Admin Functionality

Category: Broken access control | **Suggested severity:** High

The goal. The lab described a site with an unprotected admin panel, and the task was to delete the user carlos.

What I did.

1. Opened the Unprotected admin functionality lab from the Access Control section and read the description.
2. Launched the lab and added `/robots.txt` to the end of the lab URL.
3. The file listed a path I was clearly not meant to find: `/administrator-panel`.
4. Went to that path in the browser. The admin panel opened with no login prompt and no permission check.
5. Found the user carlos in the list and chose the delete option.
6. The lab confirmed the user was deleted and showed Solved.

Why it works. The developers seem to have relied on the admin URL being hard to guess. Robots.txt is a public file that tells search engines which paths to skip, so listing the admin path there actually advertises it to anyone who looks. The server itself never asked whether the visitor was an administrator.

Potential impact. Anyone, including a visitor who is not logged in, could perform administrator actions such as deleting accounts or changing data. In a real system this could lead to data loss, a full takeover of the application, or damage to other users.

Recommended fix. Require login and check the user's role on the server for every admin page and every admin action, and deny access by default. Never treat a hidden link or a robots.txt entry as protection. Admin actions should also be logged and monitored.

What I took from it: admin pages need real authentication and authorization checks, and hiding a URL in robots.txt does not secure anything.

Finding 3: File Path Traversal, Simple Case

Category: Path traversal vulnerability | **Suggested severity:** High

The goal. The task was to read the contents of the `/etc/passwd` file from the server.

What I did.

1. Opened the File path traversal, simple case lab from the Path Traversal section and read the description.
2. Launched the lab and started Burp Suite Community Edition.
3. Browsed the shop's product listings in the browser so the product images would load.
4. Opened Proxy → HTTP history in Burp and looked at the requests created while the images loaded.
5. Spotted the image requests, which carry a `filename` parameter.
6. Selected one of those image requests and changed the filename value to a directory traversal path, following the lab instructions.
7. Forwarded the modified request and checked the response.
8. Instead of an image, the server returned the contents of /etc/passwd, with entries such as root, daemon, www-data and carlos. The lab showed Solved.

Why it works. The application took the filename from the request and used it to read a file, without checking that the final path stayed inside the image folder. By adding directory traversal sequences, I could step out of that folder and reach other files on the server.

Potential impact. Depending on file permissions, an attacker could read configuration files, credentials, application data or system information. Even a read-only flaw like this often becomes the first step towards a much larger compromise.

Recommended fix. Do not use user input directly as a file path. Map the ID or name a user sends to a filename the server controls, resolve the full path and confirm it sits inside an approved base directory, and run the application with the minimum file permissions it needs.

What I took from it: weak validation of file paths can let an attacker reach files outside the folder the developers intended.

5. Evidence

I took a screenshot of each lab after completing it. Each one shows the Solved status on the PortSwigger Web Security Academy. They confirm that the labs were finished, while the observations above come from the steps I recorded as I worked.

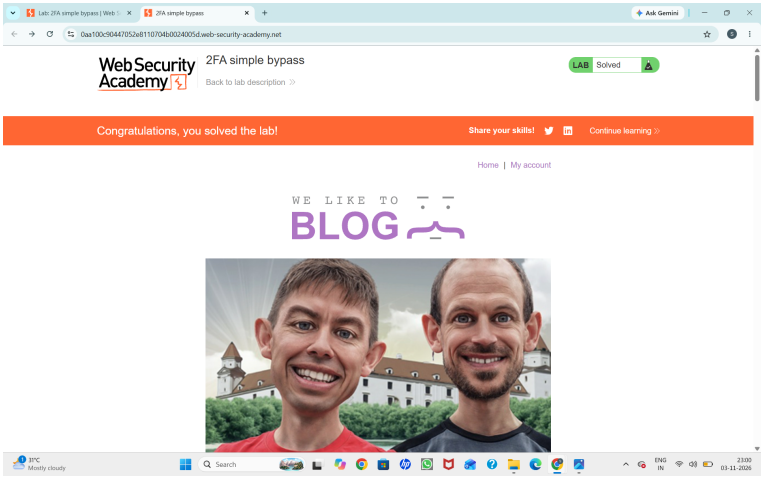


Figure 1. The 2FA simple bypass lab shown as solved.

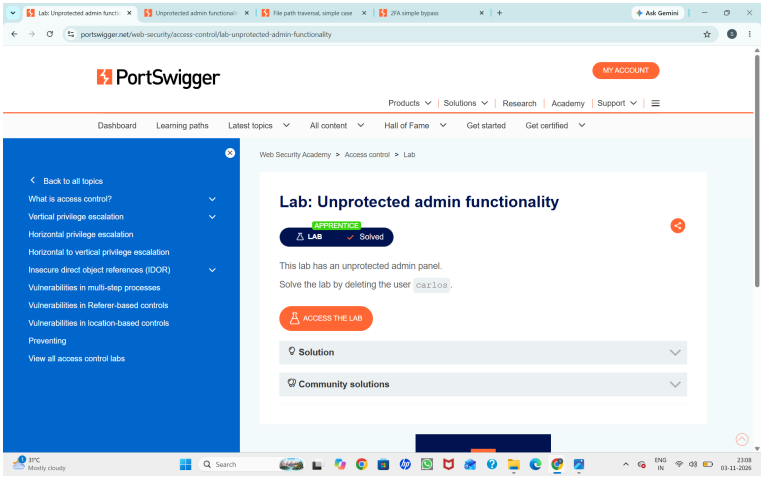


Figure 2. The unprotected admin functionality lab shown as solved.

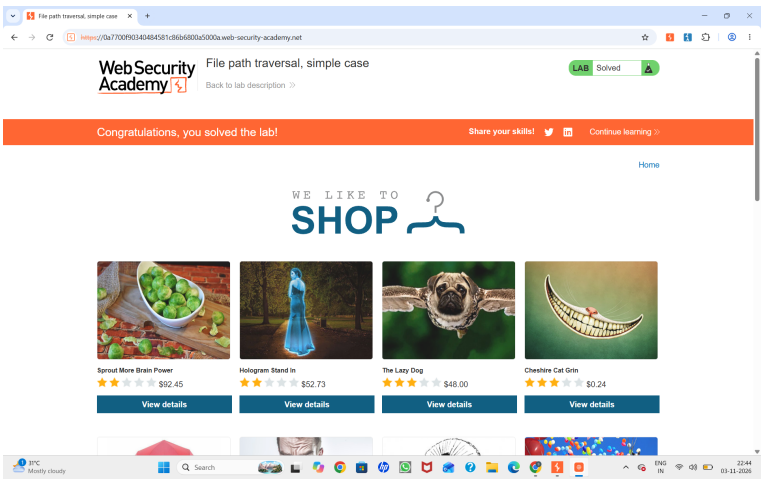


Figure 3. The file path traversal lab shown as solved.

6. Impact and Mitigation at a Glance

Vulnerability	Potential security impact	Main mitigation
2FA simple bypass	Unauthorized access to user accounts	Enforce 2FA on the server and block protected pages until verification is complete
Unprotected admin functionality	Unauthorized administrative actions	Role-based access control, with permissions checked before every admin operation
File path traversal	Exposure of sensitive server files	Validate filenames, restrict access to approved directories and use secure file handling

7. Overall Recommendations

1. Check authentication and authorization on the server for every protected page and API endpoint.
2. Only create a logged-in session once every configured authentication factor has been verified.
3. Protect admin features with role-based access control and deny access by default.
4. Never rely on hidden URLs or robots.txt to keep sensitive pages private.
5. Validate file identifiers and keep file operations inside a controlled directory.
6. Give application and operating-system accounts only the permissions they need.
7. Keep security logs, watch for unusual access patterns and test regularly.

8. Conclusion

These three labs showed me how small oversights can open up serious problems. A login flow that never enforced its second step, an admin page protected only by obscurity, and a file parameter that trusted its input each ended with access that should not have been possible.

Practically, I learned how to read and edit HTTP requests with Burp Suite, how to record evidence properly, and how to turn a finding into a clear recommendation. The common thread is that security checks must always happen on the server, on every request, and should never depend on what the user's browser does or what they are expected to find.

9. References

• PortSwigger Web Security Academy, Authentication vulnerabilities:
https://portswigger.net/web-security/authentication

• PortSwigger Web Security Academy, Access control:
https://portswigger.net/web-security/access-control

• PortSwigger Web Security Academy, Path traversal:
https://portswigger.net/web-security/file-path-traversal

Declaration. All practical work in this report was carried out only inside intentionally vulnerable PortSwigger Web Security Academy training labs, for educational purposes. No real company website or live system was tested.